CONTRAT DE DONNÉES ET D'INTERFACE

PROJET IA : IQ-PUZZLER PRO XXL (2D)

1. SYSTÈME DE COORDONNÉES ET PLATEAU

- **Dimensions de la grille 2D :** LARGEUR = 11 (colonnes x de 0 à 10), HAUTEUR = 5 (lignes y de 0 à 4).
- **Origine (0, 0) :** Coin supérieur gauche.
- **Coordonnée :** Tuple d'entiers (x, y) où 0 <= x < 11 et 0 <= y < 5.
- **État d'une case dans la grille :**
  - `None` ou `'.'` : Case vide
  - `'A'` à `'L'` (str) : Case occupée par la pièce correspondante
- **Pièces verrouillées (défi initial) :** Ensemble (set) des identifiants posés au départ.

2. REPRÉSENTATION GÉOMÉTRIQUE DES PIÈCES

- **Identifiant de pièce (id_piece) :** str parmi ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L'].
- **Forme (Orientation) :** Tuple trié de coordonnées locales normalisées à l'origine (min_x = 0, min_y = 0).
  - *Exemple pour la pièce I (3 billes) :* `((0, 0), (1, 0), (1, 1))`
- **Catalogue des domaines géométriques :**
  - Dictionnaire `{ id_piece : [liste des formes uniques après rotations/miroirs] }`

3. FORMALISATION CSP : P = (X, D, C)

- **Variables X :** Les pièces restantes à placer sur le plateau (ou la prochaine case vide selon l'heuristique MRV).
- **Domaines D :** Pour chaque pièce non posée, l'ensemble des placements valides (forme_orientee, x_ancrage, y_ancrage).
- **Contraintes C :**
    - **Unaire :** Toute bille (x_ancrage + dx, y_ancrage + dy) doit vérifier 0 <= x < 11 et 0 <= y < 5.
    - **Globale / Binaire :** Aucune superposition entre deux pièces (la case cible doit être vide).

4. SIGNATURES DES FONCTIONS INTER-MODULES

Module : pieces.py (IMAD)

```python
obtenir_catalogue_orientations() -> dict[str, list[tuple[tuple[int, int], ...]]]
```

```python
COULEURS_PIECES : dict[str, str]  # Codes couleurs Hexa officiels pour Enzo
```

Module : plateau.py (IMAD)

Classe Plateau :

```python
plateau.grille : list[list[Optional[str]]]  # (5 lignes x 11 colonnes)
```

```python
plateau.pieces_placees : dict[str, list[tuple[int, int]]]
```

```python
plateau.peut_placer(forme, x0, y0) -> bool
```

```python
plateau.poser_piece(id_piece, forme, x0, y0, verrouillee=False) -> bool
```

```python
plateau.retirer_piece(id_piece) -> bool
```

```python
plateau.premiere_case_vide() -> Optional[tuple[int, int]]
```

```python
plateau.est_complet() -> bool
```

```python
plateau.copier() -> Plateau
```

Module : solver.py (ABEL)

```python
resoudre_csp(plateau, utiliser_heuristiques=True) -> tuple[bool, Plateau, dict]
```

- *Retourne :* `(succes, plateau_resolu, metriques)`
- *Format de metriques :* `{"noeuds": int, "backtracks": int, "temps_ms": float}`

Module : defis.py (ABDELRAHIM)

```python
charger_defi(id_defi) -> dict
```

Python

```
initialiser_plateau_defi(id_defi) -> Plateau
```